ASSIGNMENT 1

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

Group MobyDick

Deep Learning Application

Instructor: Lê Thành Sách

OUR TEAM

Lê Quốc Bình – 2352117

Pipeline · CNN & Transformer · extensions

Quách Kiến Long – 2352698

Data pipeline · EDA · Linear & MLP

Lê Thanh Lâm – 2252421

LSTM & GRU · robustness study

Tô Nhã Vy – 2353360

Error analysis · interpretability · report & pages





THE PROBLEM

Task

Input: one 28×28 grayscale clothing image, tensor x of shape 1×28×28.
Output: one of 10 classes (T-shirt, Trouser, … Ankle boot).
Model $f(x) \rightarrow$ 10 logits z; prediction $\hat{y} = \text{argmax } z$.
Training: minimise cross-entropy $L = -\log \text{softmax}(z)[y]$.

Research question

How do data representation (flat vector, 2-D grid, row sequence, patch tokens) and inductive bias change accuracy, macro-F1, size and speed under one fixed protocol?
Main difficulty: look-alike classes (T-shirt / Shirt / Pullover / Coat) at very low resolution.



THE PROBLEM

HOW DIFFERENT

NEURAL NETWORK ARCHITECTURES SOLVES ONE IMAGE-CLASSIFICATION PROBLEM



Research question

How can a computer distinguish objects in an image?

How should we represent an image for learning?

How does model architecture affect learning?

Why do models make mistakes?

DATASET & EDA

Datasets

Main: Fashion-MNIST (Zalando, MIT licence) – 70,000 grayscale images, 10 classes, official 60k train / 10k test.

Debugging: no MNIST runs; sanity checks on Fashion-MNIST subsets.

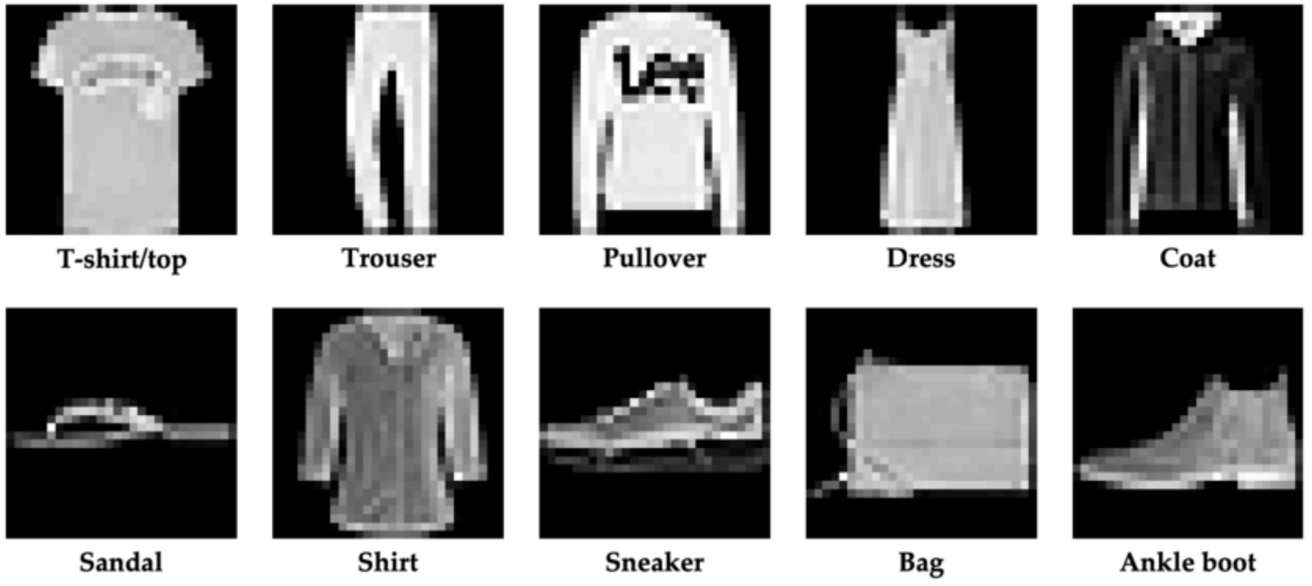
Extension: CIFAR-10 (3×32×32 RGB).

EDA findings

Class distribution: 6,000 train / 1,000 test images per class.

Imbalance: none – ratio max/min = 1.00, so accuracy and macro-F1 stay close.

Input size: every image 1×28×28, centred object, pixels 0–255.



DATASET & EDA

Datasets

Main: Fashion-MNIST (Zalando, MIT licence) – 70,000 grayscale images, 10 classes, official 60k train / 10k test.

Input size: every image 1×28×28, centred object, pixels 0–255.

Extension: CIFAR-10 (3×32×32 RGB).

EDA findings

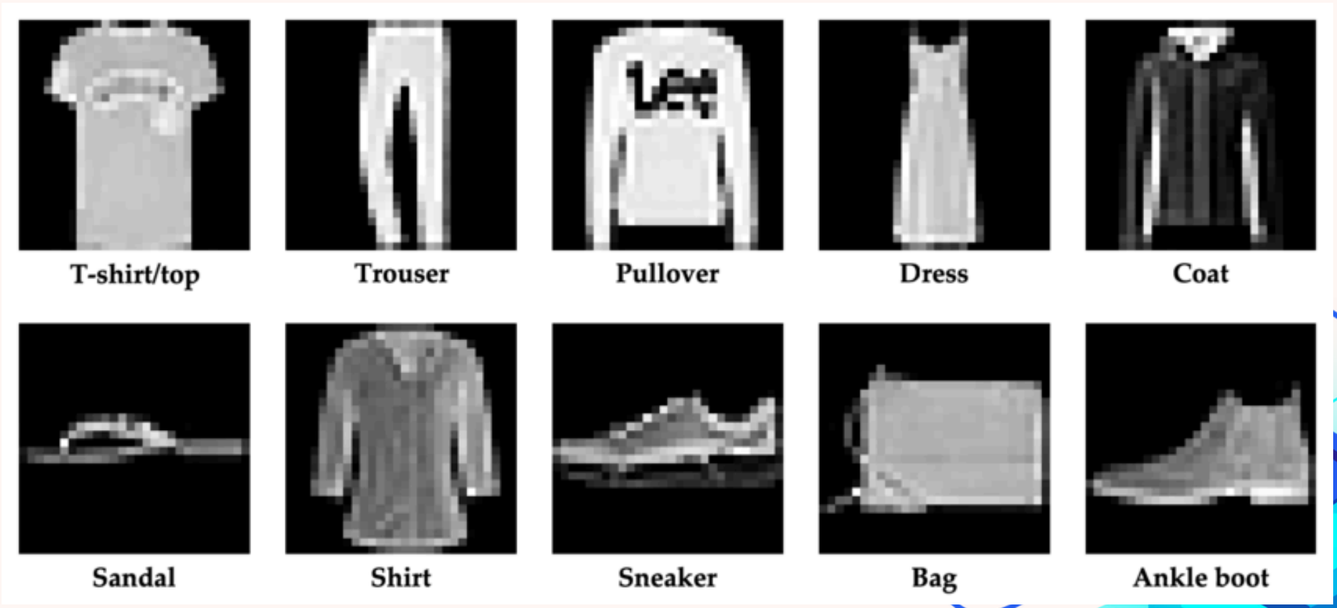
Class distribution are balanced.

Sample images can look very similar

Images are small and grayscale.

Average brightness of 0.286, and about half the pixels are black.

No completely black images or exact duplicates.



DATA PIPELINE

Split

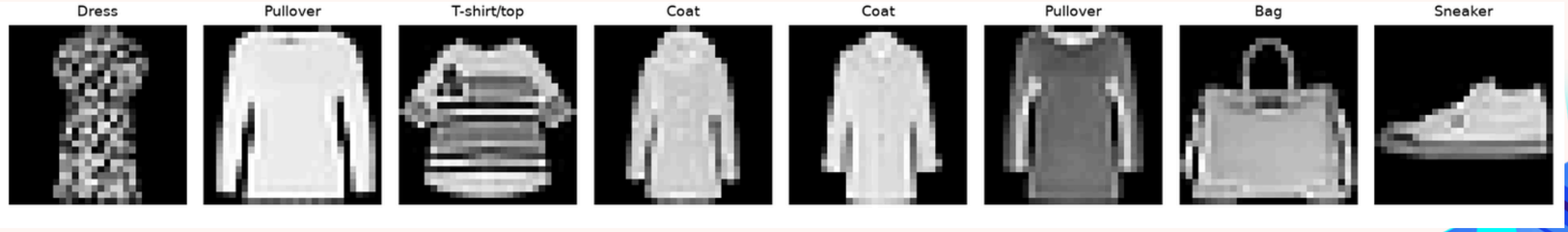
Official 60k train → random_split 50,000 train / 10,000 val (seed 42, not stratified). Official 10,000 test held out until the final checkpoint.

Preprocess

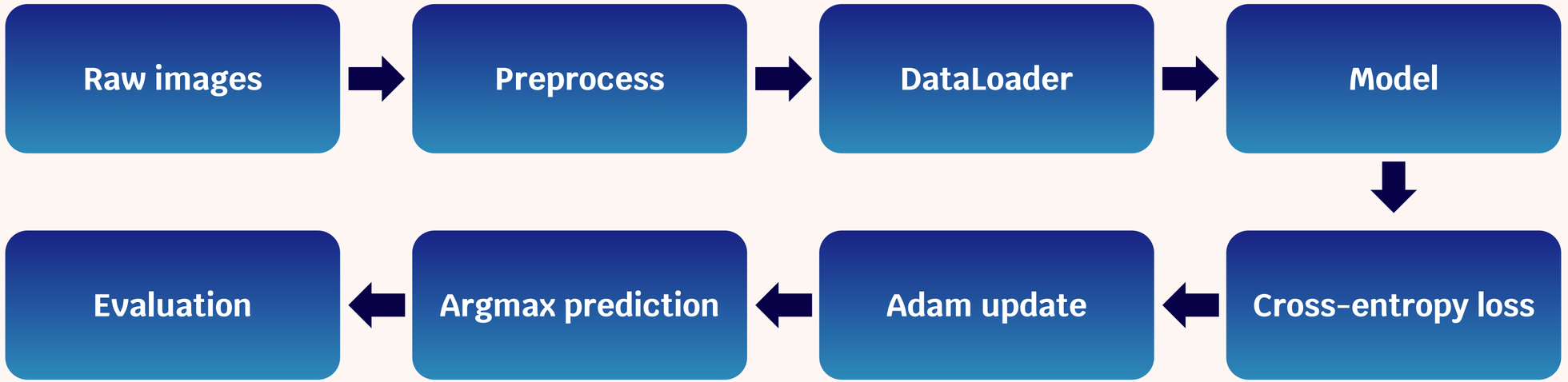
ToTensor → [0, 1]
Normalize(0.5, 0.5) → [−1, 1]
No augmentation, so every model sees identical inputs.

DataLoader

Batch size 64; train shuffled with a seeded generator.
Batch tensor: 64×1×28×28 (CIFAR-10: 64×3×32×32).



TRAINING PIPELINE



Evaluation = Accuracy, macro-F1, and inference time on the 10,000-image test set; model parameter counts and training time; training/validation curves, confusion matrices, and example predictions.

Self-implemented: data-splitting setup, training/validation/test loops, and six model architectures using PyTorch layers.

Libraries include PyTorch, torchvision, scikit-learn for metrics, and matplotlib for visualization.

MODELS

Flattened pixels:

LINEAR

MLP

Spatial structure:

CNN

Row sequence:

LSTM

GRU

Attetion:

Transformer



FAIR SETUP

Identical for every model

Same split & seed 42 (Python, NumPy, PyTorch, CUDA; cuDNN deterministic) · Adam, lr 1e-3 · batch 64 · 10 epochs · CrossEntropyLoss · no scheduler, no early stopping.

Checkpoint rule & measurements

Keep the epoch with the best validation macro-F1, reload it once for the test set. Report accuracy, macro-F1, #params, total training time, inference time on 10k test images.
Run environment: laptop CPU and GPU (Intel Core Ultra 9 275HX, RTX 5060 GPU), Windows 11



LINEAR & MLP

Linear / softmax

Flatten 1×28×28 → 784-vector → Linear(784, 10). Logits go straight into CrossEntropyLoss – no softmax layer before it.

Params: 7,850

MLP

784 → 256 (BatchNorm, ReLU, Dropout 0.2) → 128 (ReLU, Dropout 0.2) → 10.

ReLU adds non-linearity; BatchNorm and dropout regularise.

Params: 235,658

Inductive bias

Both treat the image as an unordered vector of 784 pixels. They have no built-in spatial locality or translation invariance, so shifting an object changes which input positions receive its features.



CNN

Our architecture

Conv 3×3 (1→16) + BN + ReLU +
MaxPool 2×2 → 16×14×14
Conv 3×3 (16→32) + BN + ReLU +
MaxPool 2×2 → 32×7×7
Flatten 1,568 → FC 128 → Dropout 0.3 → 10
Params: 207,018 (≈97% in the FC head)

Why it fits images

Convolution: the same 3×3 kernel slides over the whole image → local, translation-equivariant features with few weights.
Pooling: 2×2 max keeps the strongest response, halves resolution (28→14→7) and tolerates small shifts.
Feature maps: each channel is a learned detector – edges in block 1, textures and parts in block 2.

Inductive bias

Locality and weight sharing: Learns local patterns and detects them across different image locations.



LSTM & GRU

Image → sequence

Each row is one timestep.
28 timesteps × 28 features (C·W).
Input tensor: B×28×28 (CIFAR-10: B×32×96).

Recurrent core

2 layers, hidden size 128, dropout 0.2.
Last hidden state h28 → BatchNorm → Dropout → Linear(128→10).
Params: LSTM 214,538 · GRU 161,290 (3 gates vs 4)

Hidden state

h_t summarises rows 1…t of the image.
Only h28 is classified, so information from the top rows must survive 28 gated steps.
Order is modelled along height only.

Inductive bias

Sequential processing and memory: Reads rows in order and carries earlier information forward to classify the image.



TRANSFORMER

Patch tokens

4×4 patches → 7×7 = 49 tokens.
Patch projection: Conv2d(k=4, stride=4) → 64-d embedding per patch.
Prepend a learnable [CLS] token → 50 tokens.
Add a learnable positional embedding (50×64).

Encoder & attention

Encoder: 3 layers, 4 heads, MLP 128, GELU, dropout 0.1 · 105,546 params.
Attention in: 50×64 tokens → queries, keys, values (16-d per head).
Attention out: 50×64 – every token becomes a weighted mix of all tokens, so global context exists from layer 1.
Head: LayerNorm on [CLS] → Linear(64→10).

Inductive bias

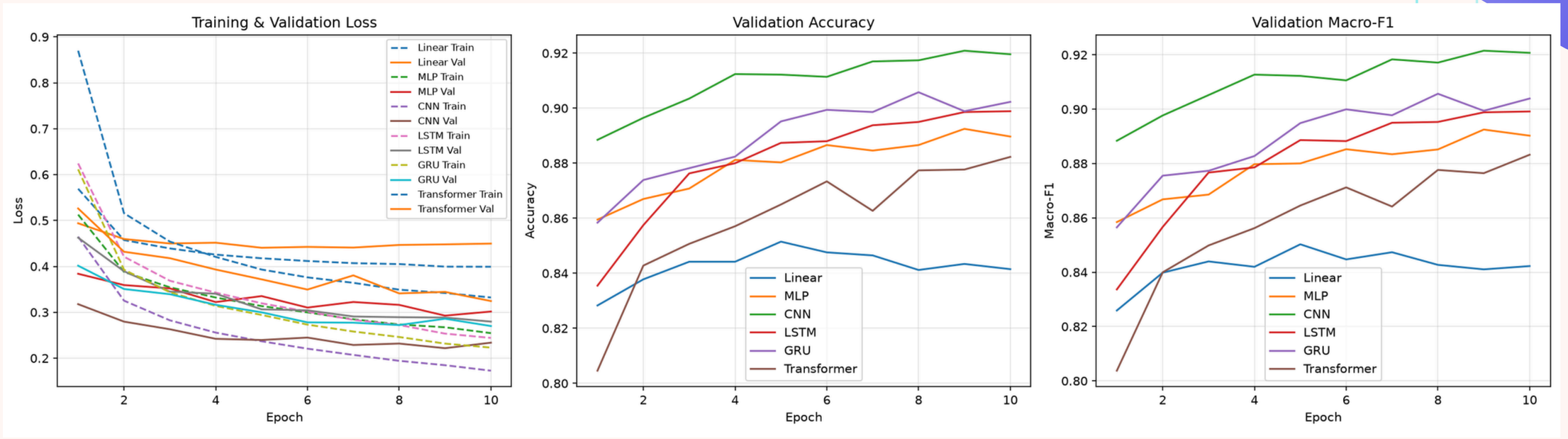
Patch-based representation and global attention: Groups pixels into patches and learns relationships across the image, using positional embeddings to retain location information.

RESULTS

Model	Parameters	Best Acc	Best F1	Test Acc	Test Macro F1	Train Time (s)	Inference (s)
Linear	7850	0.8515	0.8503	0.837	0.8353	27.42	0.42
MLP	235658	0.8925	0.8925	0.8855	0.8846	30.11	0.44
CNN	207018	0.9209	0.9215	0.9122	0.912	34.7	0.5
LSTM	214538	0.8989	0.8992	0.8922	0.8919	35.06	0.53
GRU	161290	0.9058	0.9057	0.8966	0.8957	33.37	0.52
Transformer	105546	0.8823	0.8832	0.8815	0.8818	54	0.53

Key takeaway: CNN is best on both metrics (91.22% / 0.912), +1.6 pts over GRU and +2.7 over MLP with fewer parameters than MLP or LSTM, and trains in 34.7 s. The Transformer is the smallest model but the slowest to train Transformer is the smallest nonlinear model, but the slowest to train (53.78 seconds, approximately 1.55× CNN), reaching 88.15% accuracy. Linear remains the smallest and fastest overall.
Accuracy and macro-F1 give the same ranking.

LEARNING CURVES



Overfitting

CNN train loss keeps falling (≈0.17) while val loss flattens at ≈0.22–0.24 → best val F1 at epoch 9.

Plateau

Linear peaks at epoch 5 (val F1 0.850) and then drifts – limited by capacity, not by epochs.

Under-trained

LSTM and Transformer are still improving at epoch 10 → both need more epochs.

ERROR ANALYSIS

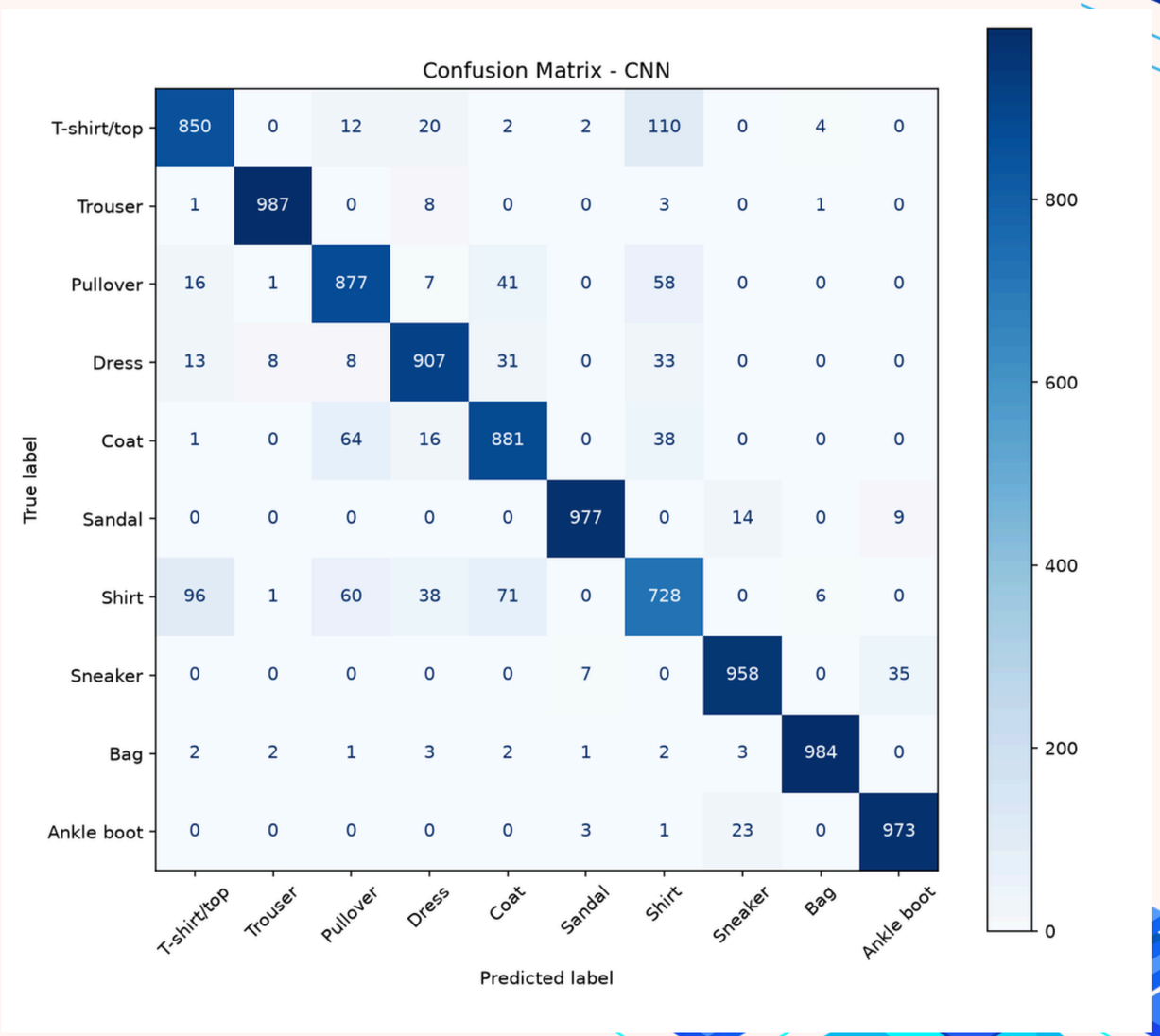
CNN: 878 test errors

1. Tops and dresses – 735 (83.7%)
T-shirt → Shirt: 110; Shirt → T-shirt: 96; Shirt → Coat: 71; Coat → Pullover: 64. Shirt recall: 72.8%.

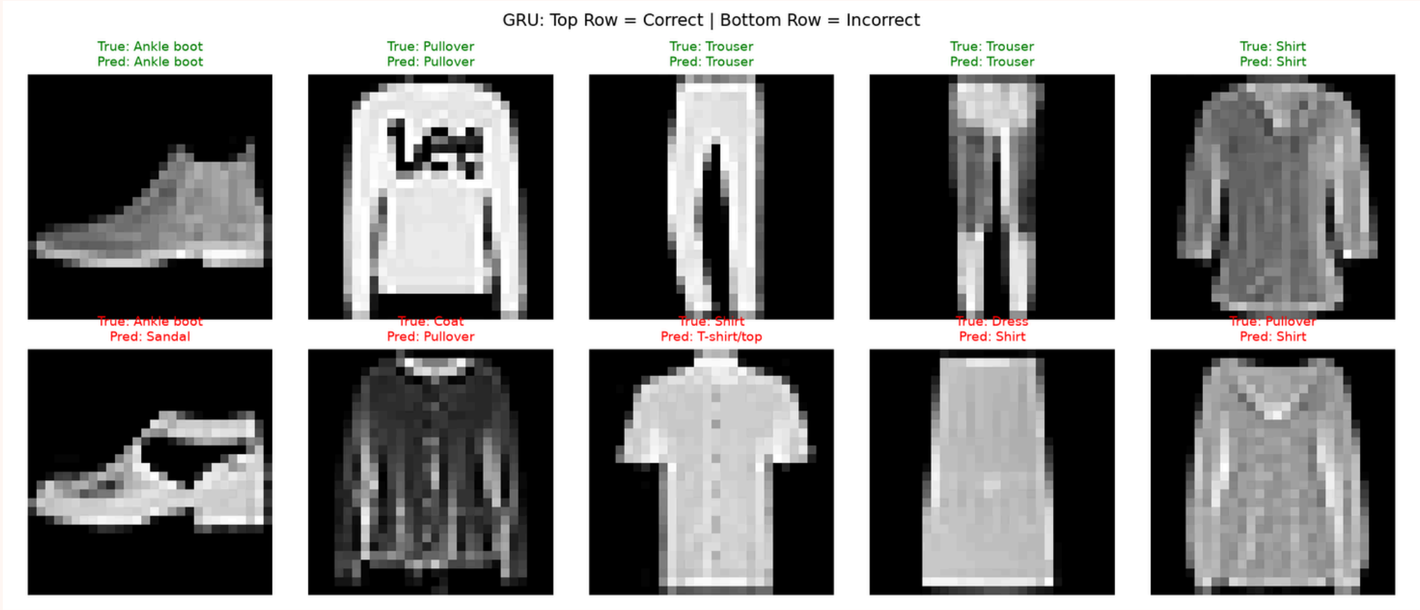
2. Footwear categories – 91 (10.4%)
Sneaker → Ankle boot: 35; Ankle boot → Sneaker: 23; other footwear confusions: 33.

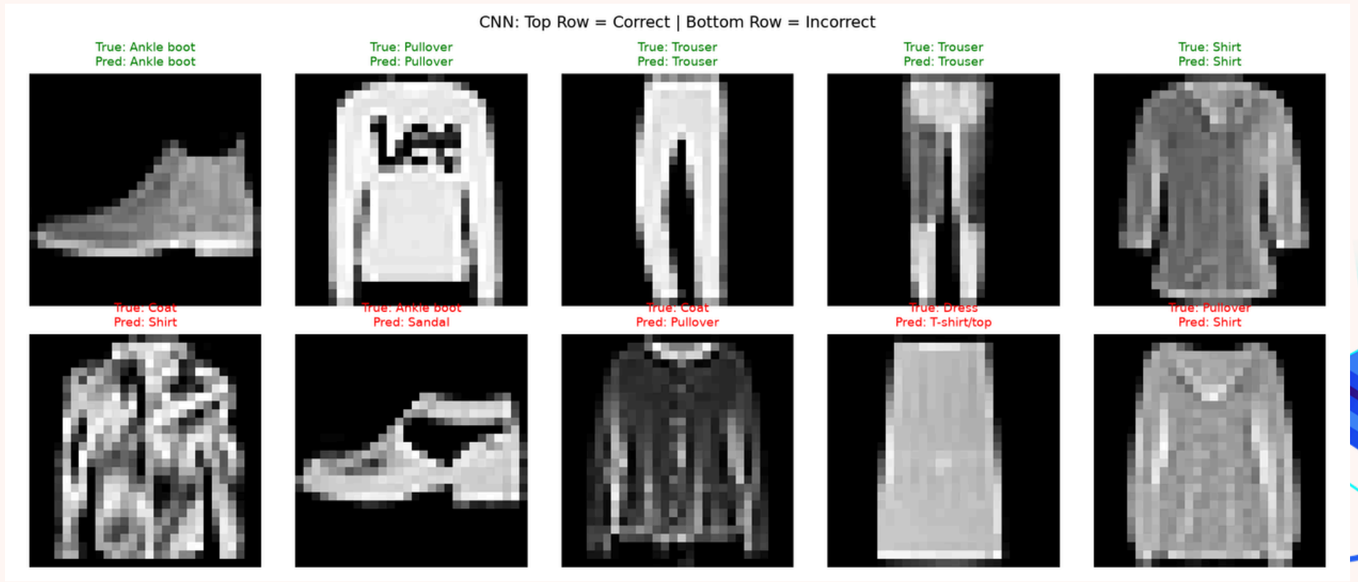
3. Remaining errors — 52 (5.9%)
Includes 16 misclassified bags, 13 misclassified trousers, and 23 other errors outside the two groups above.

Ideas: flip/shift augmentation, more early-layer channels, higher resolution.



PREDICTIONS





WHY DO MODELS PERFORM DIFFERENTLY?

Representation → result

Flat vector: Linear 83.70%, MLP 88.55% – no locality.

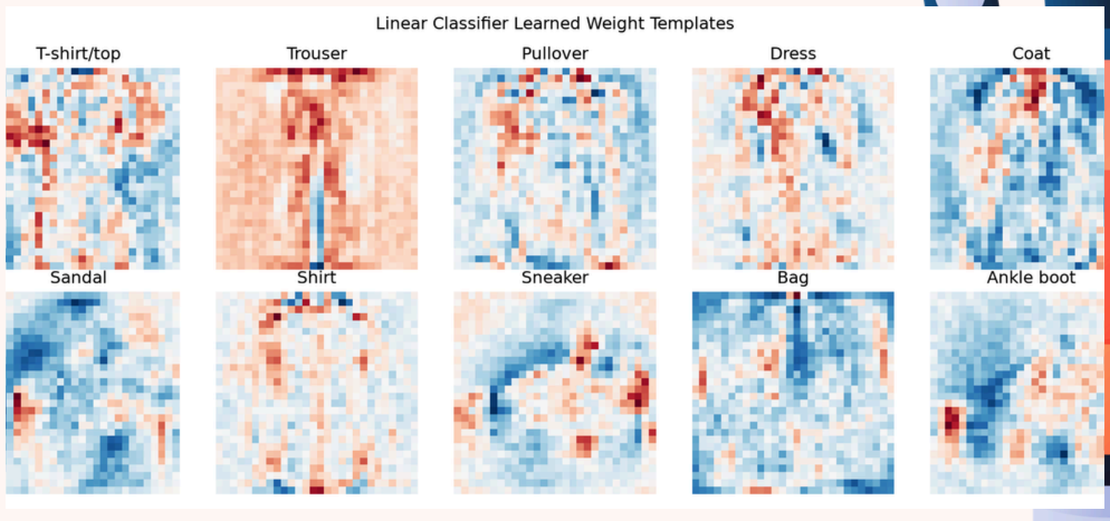
2-D grid: CNN 91.22% – locality + weight sharing fit garments; best under occlusion.

Row sequence: GRU 89.66% / LSTM 89.22% – most noise-robust.

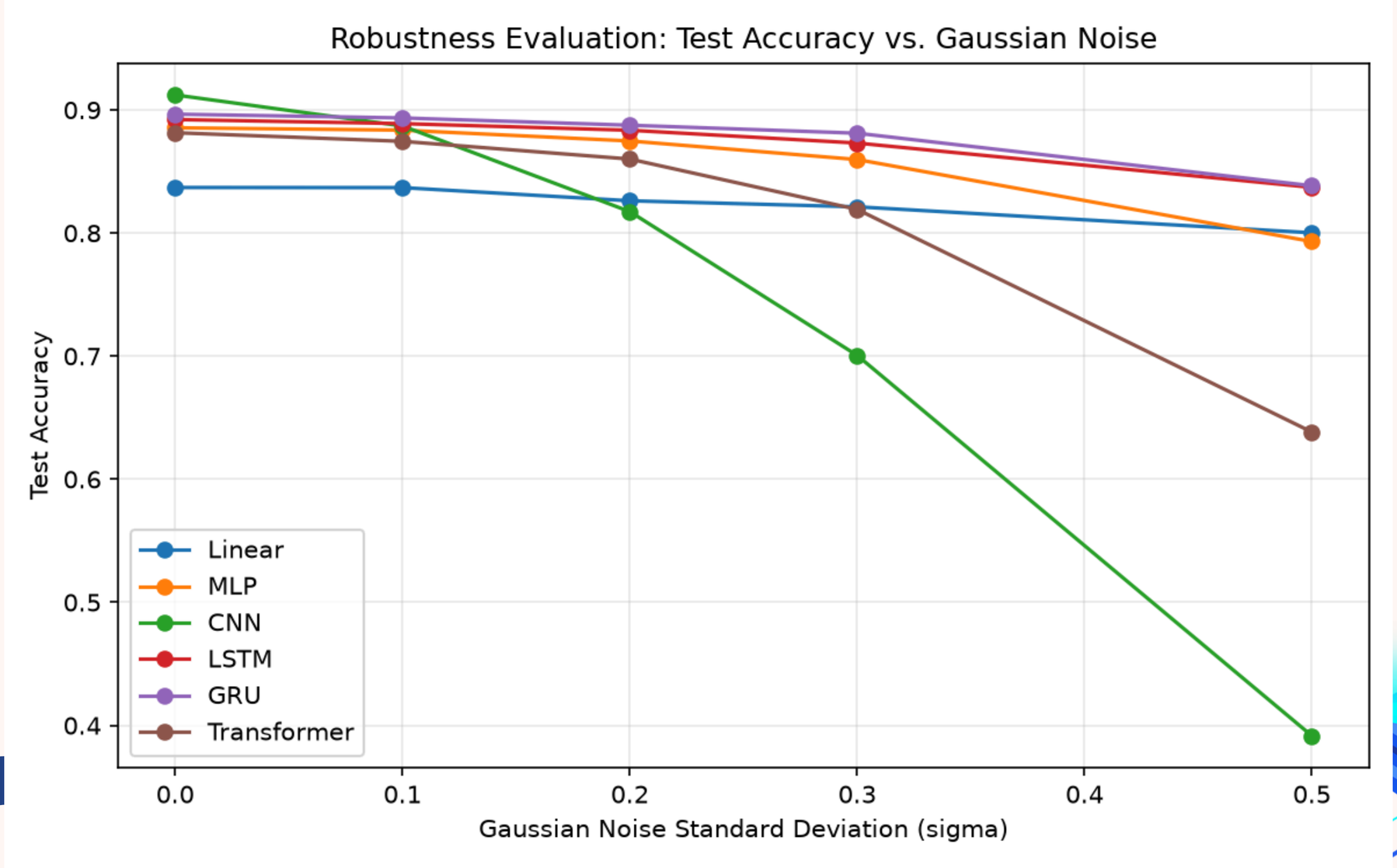
Patches: ViT 88.15% – weakest prior, still improving at epoch 10.

Interpretability

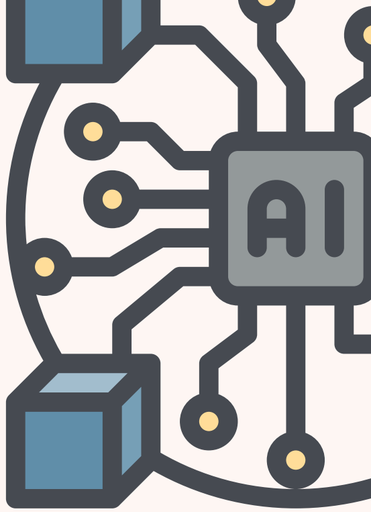
Linear weights are blurry class templates (trouser legs, bag handle, boot shape): the linear model does template matching.



ROBUSTNESS


Robustness Evaluation: Test Accuracy vs. Gaussian Noise
Test Accuracy
0.9
0.8
0.7
0.6
0.5
0.4
0.0
0.1
0.2
0.3
0.4
0.5
Gaussian Noise Standard Deviation (sigma)
Linear
MLP
CNN
LSTM
GRU
Transformer

CONCLUSION



Takeaways

CNN gives the best trade-off: 91.22% acc / 0.912 macro-F1 with 207K params – its locality prior matches clothing images.

Sequence models are 1.6 pts behind but far more noise-robust; the ViT needs more epochs or data.

Errors concentrate on look-alike tops (84%).

Limitations & next

Single seed, 10 epochs, no augmentation or LR schedule; non-stratified split; extensions from a separate CPU run; timings include data loading.

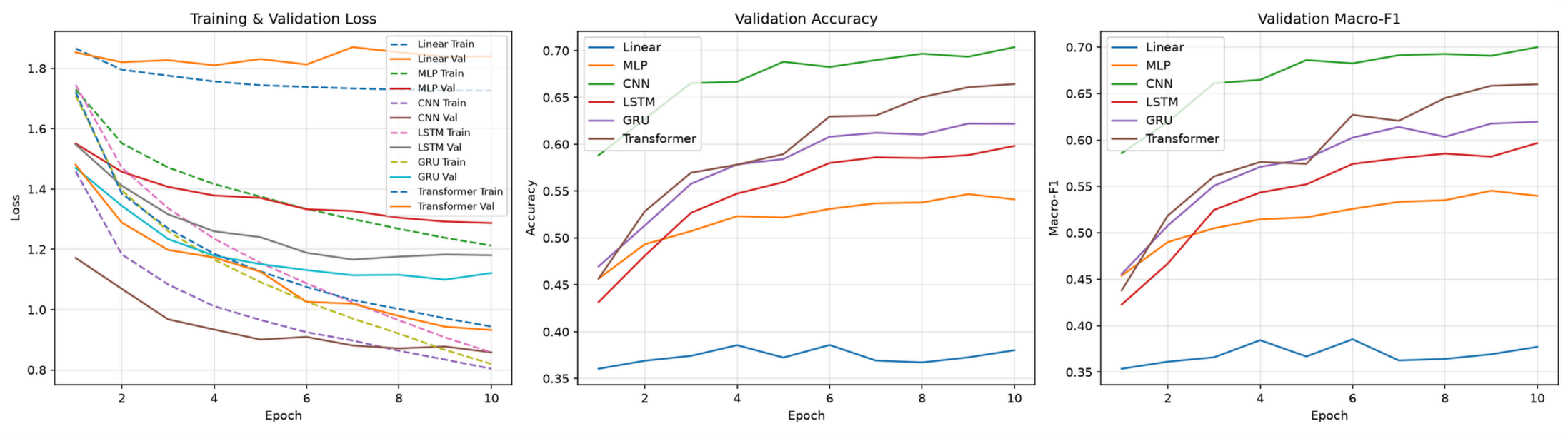
Next: mean ± std over seeds, augmentation, longer ViT training, truly matched budgets.

EXTENSION RESULTS

Model	Parameters	Best_Val_Acc	Best_Val_F1	Test_Acc	Test_Macro_F1	Train_Time_s	Inference_Time_s
Linear	30730	0.3858	0.3854	0.383	0.3827	29.82	0.56
MLP	821386	0.5468	0.5453	0.5479	0.5456	32.5	0.55
CNN	268746	0.7036	0.6999	0.6997	0.6968	33.6	0.54
LSTM	249354	0.5982	0.5966	0.606	0.6053	36.04	0.6
GRU	187402	0.622	0.6196	0.6179	0.6166	34.43	0.57
Transformer	108554	0.6642	0.6599	0.6628	0.6601	60.03	0.69

Key takeaway: CNN leads on CIFAR-10 with 69.97% accuracy / 0.6968 macro-F1, training in 33.60 s. Transformer ranks second at 66.28%, trailing CNN by 3.69 percentage points while taking 1.79× as long to train. GRU outperforms LSTM with fewer parameters and shorter training time. MLP has the most parameters but reaches only 54.79%, showing that more parameters do not guarantee better performance. Accuracy and macro-F1 give the same ranking.

LEARNING CURVES



Learning progress

CNN and Transformer improve across ten epochs; validation accuracy reaches 70.4% and 66.4%.

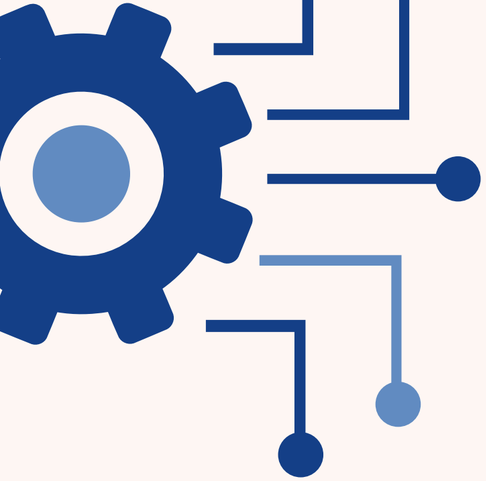
Signs of overfitting

LSTM/GRU training losses keep falling while validation losses flatten. MLP shows a smaller growing gap.

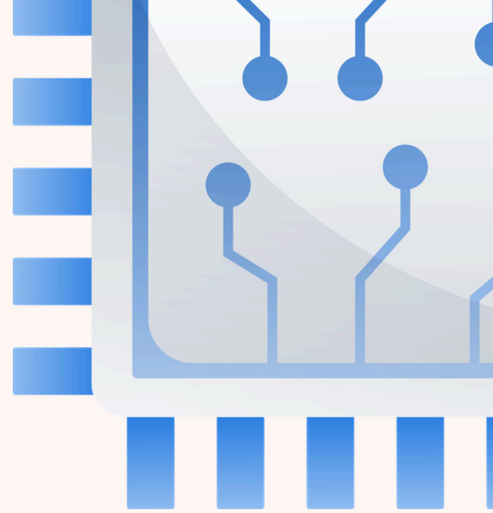
Plateau

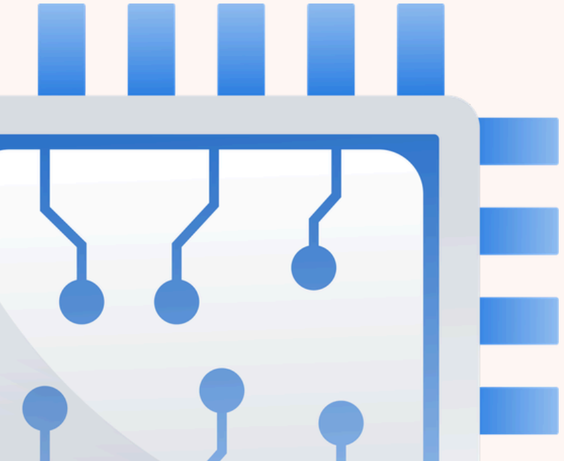
Linear validation accuracy fluctuates around 36–39%, with little sustained improvement.

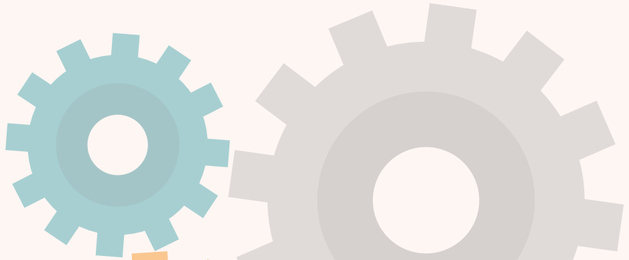
THANK YOU













QUESTION TIME

